

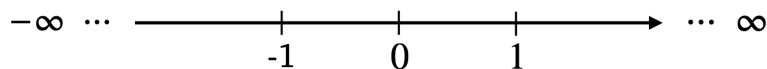
Introduction to the floating point representation

2025年9月1日 8:09

Lecture Outline

- Floating-point number systems
- Rounding
- Machine epsilon
- IEEE 754 standard

Floating Point Recap



The radix can "float" between digits

e.g. 6 digits, we can repres: 0.123456

0.00123456

123456.00

The position of the radix is determined from an exponent, e.g. $0.123456 = 1.23456 \times 10^{-1}$
 $= 1234.56 \times 10^{-4}$

To ensure uniqueness of repres, assume first digit is non-zero, and followed by radix

A FP system is characterized by four integers, (B, t, L, U) .

B is the base of the system or the radix; $B = \{2, 10\}$

t is number of digits or precision.

$[L, U]$ is the exponential range

A number x belongs to FP is represented as:

$$x = \pm \left(d_0 + \frac{d_1}{\beta_1} + \frac{d_2}{\beta_2} + \dots + \frac{d_{t-1}}{\beta_{t-1}} \right) \cdot \beta^e$$

$$0 \leq d_i \leq \beta - 1, i = 0 \dots t-1$$

$e \in [L, U]$ is an integer

The string of t digits $d_0 d_1 \dots d_{t-1}$

is called mantissa or significand

$d_1 d_2 \dots d_{t-1}$ is called fractional

A common way to write $x \in \text{FP}$ is:

$$x = \pm d_0 . d_1 d_2 \dots d_{t-1} \times \beta^e$$

$\hookrightarrow$ radix point

A FP number is called *normalized* if $d_0 \neq 0$, and is *denormalized* if $d_0 = 0$.

Floating Point Number Systems

Example 1: Consider the FP system ($\beta = 10, t = 3, L = -2, U = 2$).

Numbers in system: $d_0.d_1d_2 \times 10^e$

Largest: 9.99×10^2

Smallest: 0.01×10^{-2}

Denormalized numbers include: 0.23×10^{-2}

Zero is: 0.00×10^0

Lecture Outline

- Floating-point number systems
- Rounding
- Machine epsilon
- IEEE 754 standard

Rounding

rounding

How do we store a real number

$$x = \pm d_0.d_1 \cdots d_{t-1}d_t d_{t+1} \cdots \times \beta^e$$

in t digits?

Denote by $fl(x) : \mathbb{R} \rightarrow \mathcal{FP}$ the "rounding" function.

Round by "chopping" (rounding towards 0)

Round to nearest: $fl(x)$ is nearest $\mathcal{FP}$

Round to $+\infty$, $fl(x)$ is the smallest $\mathcal{FP} \geq x$

$-\infty$: $fl(x)$ is the largest $\mathcal{FP} \leq x$

Rounding

Example 2: Consider again the FP system ($\beta = 10$, $t = 3$, $L = -2$, $U = 2$). Let $x = 1.2789 \times 10^1$.

- Chopping: 1.27×10^1
- Nearest: 1.28×10^1
- $+\infty$: 1.28×10^1
- $-\infty$: 1.27×10^1

Let $x = 1.275000 \times 10^1$. It is in the middle of 1.27 and 1.28. When there's a tie, convention is to round to the number with the last even digit: 1.28×10^1

Lecture Outline

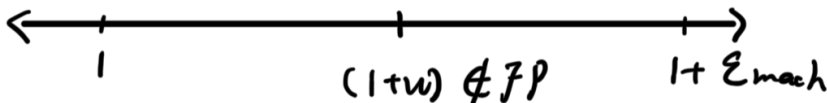
- Floating-point number systems
- Rounding
- Machine epsilon
- IEEE 754 standard

Machine Epsilon

Machine epsilon (ϵ_{mach}): the distance from 1 to the next larger FP number.

Eg. for $\tau=5$, $\beta=10$, $\epsilon_{\text{mach}} = |1 - 1.0001|$
 $= 10^{-4}$

unit roundoff: $u = \frac{\epsilon_{\text{mach}}}{2}$



Equiv def: u is the smallest positive number
such that $fl(1+\varepsilon) > 1$

When rounding to nearest, $fl(x) = x(1+\varepsilon)$, where
 $|\varepsilon| \leq u = \frac{\varepsilon_{mach}}{2}$

$$\text{i.e. } \frac{fl(x) - x}{x} = \varepsilon, \quad \left| \frac{fl(x) - x}{x} \right| = |\varepsilon| \leq u$$

Lecture Outline

- Floating-point number systems
- Rounding
- Machine epsilon
- IEEE 754 standard

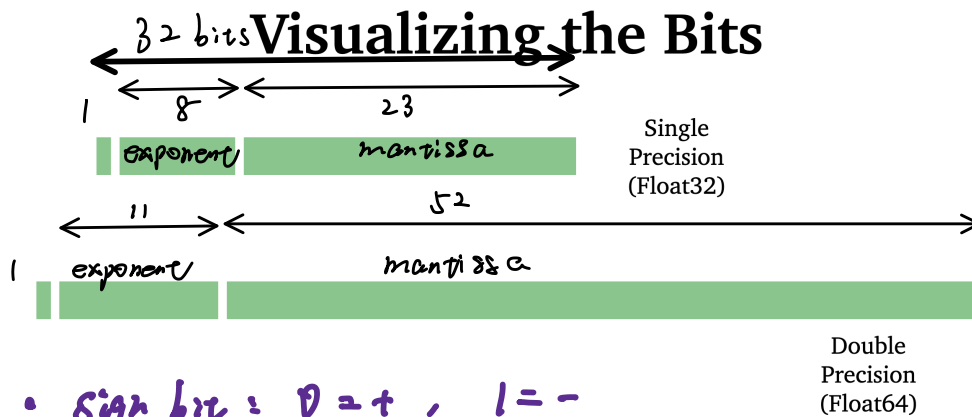
IEEE 754 Standard

- Standard for floating point arithmetic (1985)
- IEEE 754-2008 and IEEE 754-2019 are updated versions
- Most common binary representations are **single** (32 bit) and **double** (64 bit) precision; since 2008 there is also **half** (16 bit) precision

	bits	t	L	U	ϵ_{mach}
single	32	24	-126	127	$\approx 1.2 \times 10^{-7}$
double	64	53	-1022	1023	$\approx 2.2 \times 10^{-16}$

	range	normalized	denormalized
single	$\pm 3.4 \times 10^{38}$	$\pm 1.2 \times 10^{-38}$	$\pm 1.4 \times 10^{-45}$
double	$\pm 1.8 \times 10^{308}$	$\pm 2.2 \times 10^{-308}$	$\pm 4.9 \times 10^{-324}$

IEEE 754 - Exceptional Values



- sign bit: 0 = +, 1 = -
- exponent: unsigned and biased integer, the bias puts us in the range $[L, U]$
- first bit of mantissa is not stored, it's assumed to be 1 (sticky bit) for normalized numbers.

一般来说，系统的bias就是U的值，对于e的bit数，先用 $L - U + 1$ 找出一共需要表示多少个指数，然后求解 $2^N \geq L - U + 1$ ，找最小满足要求的N。